

15 Graph Theory and Network



15.2 Directed Graphs

Edges have direction, so connections are no longer symmetric; an asymmetric adjacency matrix describes a directed graph, and its powers count paths through intermediate nodes

Linear Algebra Made Easy and Visual | Open-source code & drafts: <https://github.com/Visualize-ML>

Core Skills You Will Master

1

A directed graph consists of nodes and directed edges; each edge encodes a one-way relationship

3

Powers of the adjacency matrix: entry (i, j) of the k -th power counts paths from i to j using exactly k edges

5

Use the adjacency matrix and its powers to solve a real problem: the classic ball-passing problem

2

The adjacency matrix of a directed graph is a square matrix that is generally not symmetric: entry (i, j) marks an edge from node i to node j

4

Out-degree and in-degree: row sums of the adjacency matrix give out-degree, column sums give in-degree

From directed graphs to adjacency matrices, degree, and the ball-passing problem

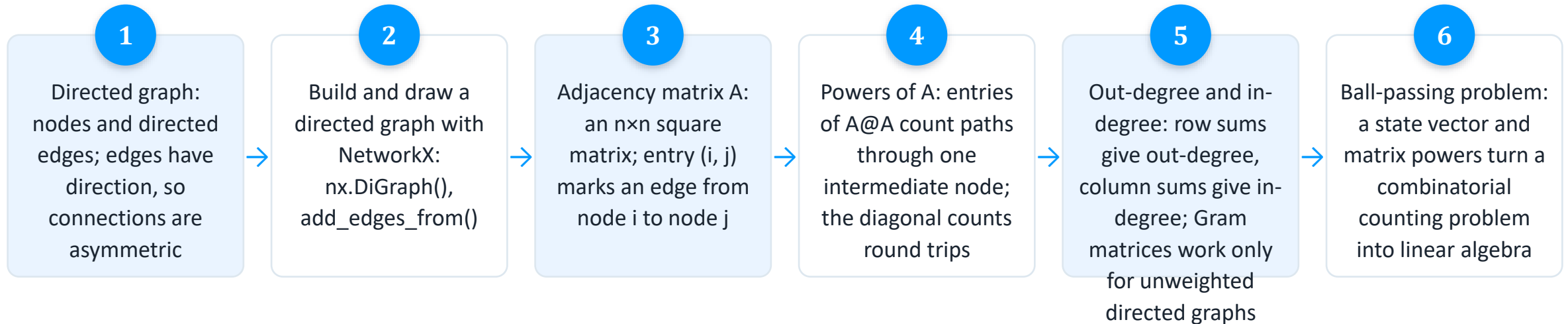
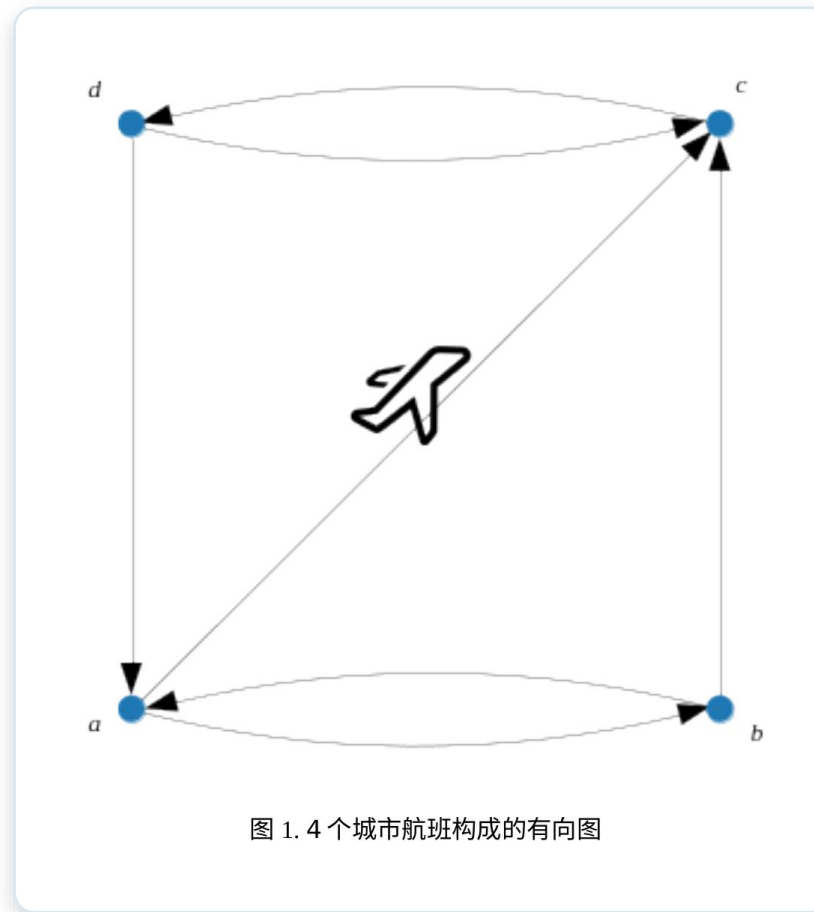


Figure 1 · A Directed Graph Formed by Flights Among 4 Cities

- Treat 4 cities a, b, c, d as nodes and flights as directed edges; round-trip flights exist between a, b and between c, d , but only a one-way flight from a to c — direction cannot be ignored



Building and Drawing a Directed Graph with NetworkX

Code 1. Drawing a directed graph with NetworkX

- `nx.DiGraph()` creates a directed graph object; `add_nodes_from()` adds several nodes at once
- `add_edges_from()` adds several directed edges at once; ('a','b') means an edge from a to b, a different edge from ('b','a')
- The dictionary `pos` manually sets each node's 2D coordinates for a custom layout
- `connectionstyle='arc3, rad=0.1'` in `draw_networkx()` bends bidirectional edges into curved arcs, so two overlapping straight lines don't hide each other's direction

```
## 初始化
import matplotlib.pyplot as plt
import networkx as nx

## 有向图
a directed_G = nx.DiGraph()

## 节点
b directed_G.add_nodes_from(['a', 'b', 'c', 'd'])

## 有向边
c directed_G.add_edges_from([('a', 'b'),
                             ('b', 'a'),
                             ('a', 'c'),
                             ('d', 'a'),
                             ('b', 'c'),
                             ('c', 'd'),
                             ('d', 'c')])

## 定义节点坐标
d pos = {'a': [0, 0],
         'b': [1, 0],
         'c': [1, 1],
         'd': [0, 1]}

## 可视化
e plt.figure(figsize = (6,6))
nx.draw_networkx(directed_G, pos = pos, node_size = 188,
                 connectionstyle = 'arc3, rad = 0.1')
```

A Directed Graph Can Be Represented by an Adjacency Matrix (Usually Not Symmetric)

$$A = [0 \ 1 \ 1 \ 0; 1 \ 0 \ 1 \ 0; 0 \ 0 \ 0 \ 1; 1 \ 0 \ 1 \ 0]$$

- For a directed graph with n nodes, the adjacency matrix is an $n \times n$ square matrix A ; entry (i, j) marks the number (or weight) of edges from node i to node j
- Note the order: row first, then column — an edge from a to b sets A 's row 1, column 2 entry to 1
- Every undirected graph's adjacency matrix is symmetric; in contrast, a directed graph's adjacency matrix is generally not

Figure 2 · Every Entry of a Directed Graph's Adjacency Matrix

- Check each entry against its edge one by one: (a) and (b) correspond to $a \rightarrow b$ and $a \rightarrow c$; (c)–(g) correspond in turn to $b \rightarrow a$, $b \rightarrow c$, $c \rightarrow d$, $d \rightarrow a$, $d \rightarrow c$

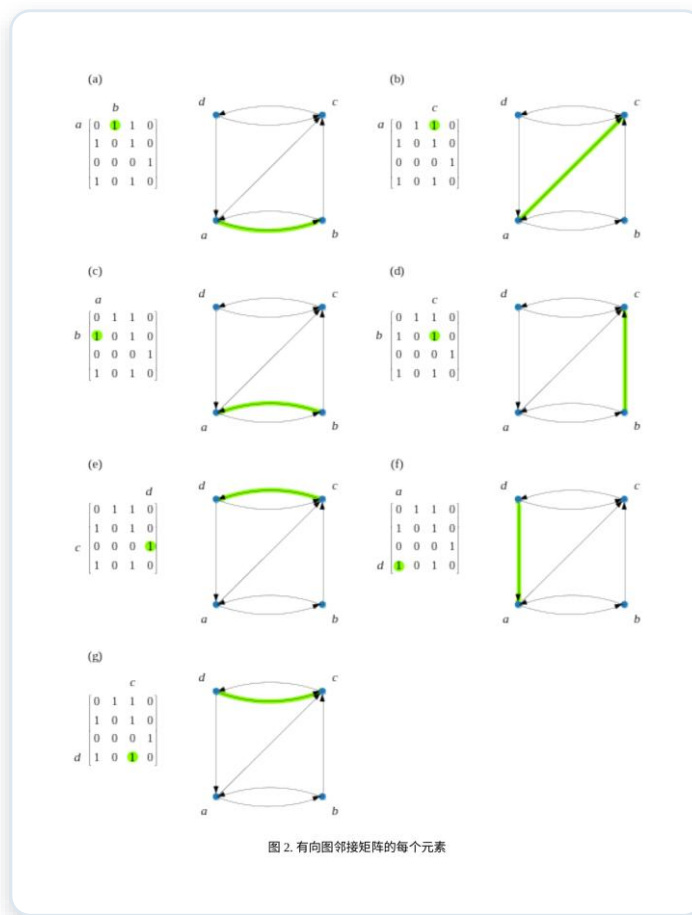


Table 1 · Several Directed Graphs on 4 Nodes and Their Adjacency Matrices (Top)

- The same 4 nodes a, b, c, d can form directed graphs with very different structures, each with its own asymmetric adjacency matrix; work through each pairing

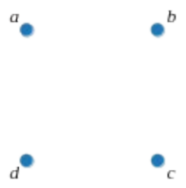
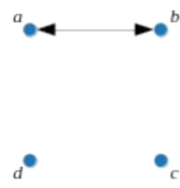
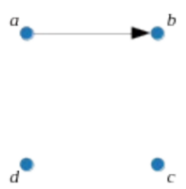
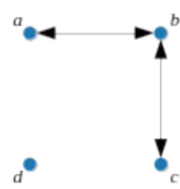
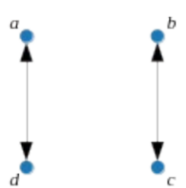
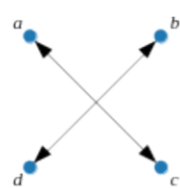
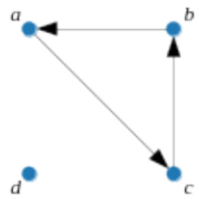
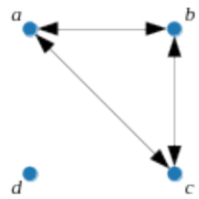
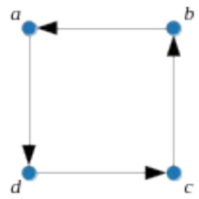
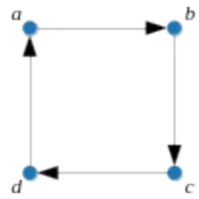
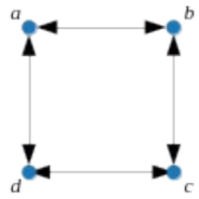
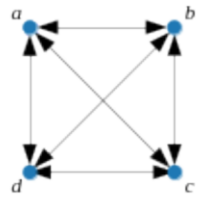
有向图	邻接矩阵	有向图	邻接矩阵
	$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$		$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$
	$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$		$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$
	$\begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$		$\begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$

Table 1 · Several Directed Graphs on 4 Nodes and Their Adjacency Matrices (Continued)

- Continue analyzing the remaining rows; notice how two-way and one-way edges show up differently in the matrix

	$\begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$		$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$
	$\begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$		$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix}$
	$\begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix}$		$\begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$

Figures 3–4 · Nodes a Can Reach Directly, Nodes That Can Reach a Directly

- Row 1 of A (value 1) marks the nodes a reaches directly without passing through any node, b and c , written $[1\ 0\ 0\ 0]@A$; column 1 marks the nodes that reach a directly, b and d , written $A@[1\ 0\ 0\ 0]^T$

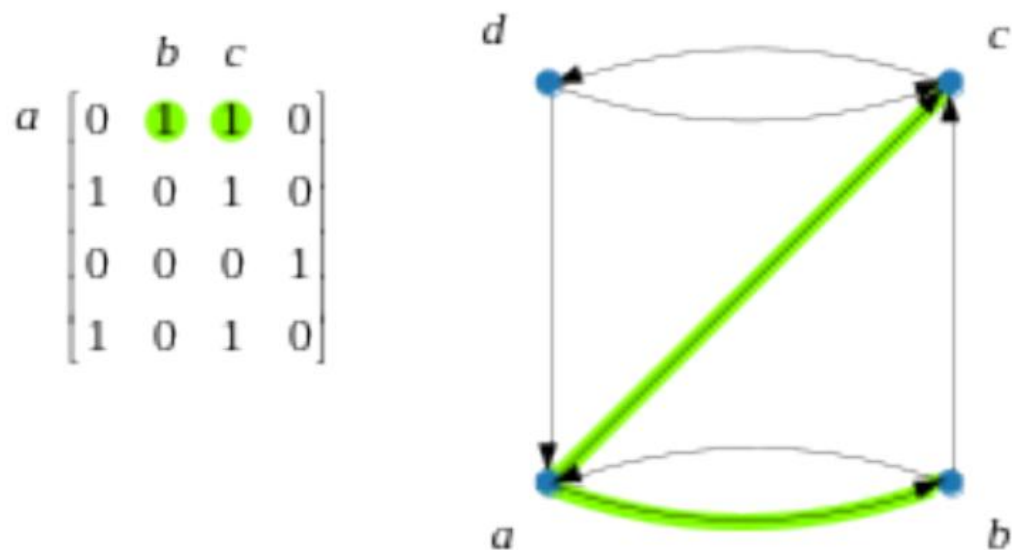


图 3. a 直达两个节点，有向图邻接矩阵 A 的第 1 行

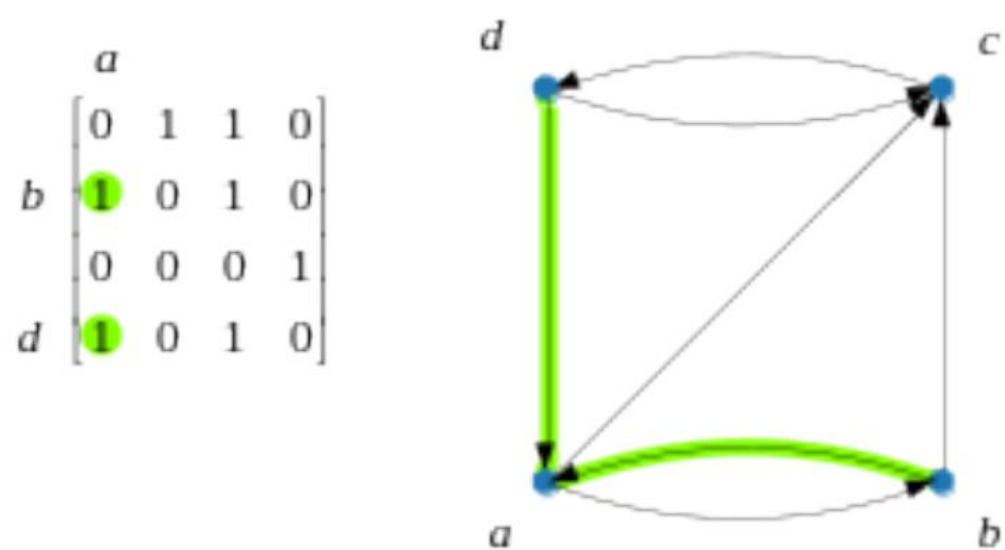


图 4. 直达 a 的两个节点，有向图邻接矩阵 A 的第 1 列

Figures 5–6 · a-to-c and c-to-a Paths Through One Intermediate Node

- Entry (1,3) of $A@A$ is 1, matching the path $a \rightarrow b \rightarrow c$ through one node; entry (3,1) of $A@A$ is 1, matching the path $c \rightarrow d \rightarrow a$

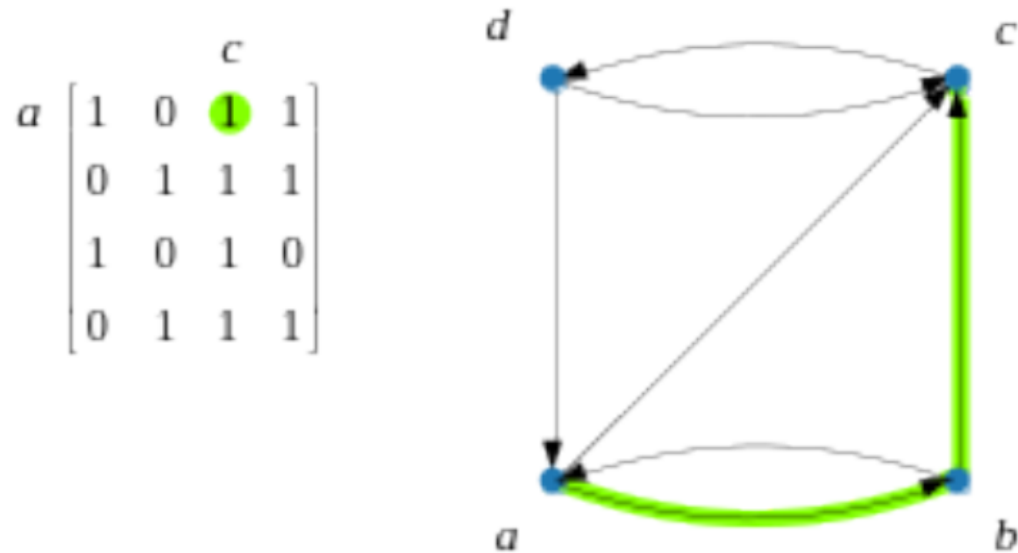


图 5. 节点 a 途径一个节点到达 c 的路径

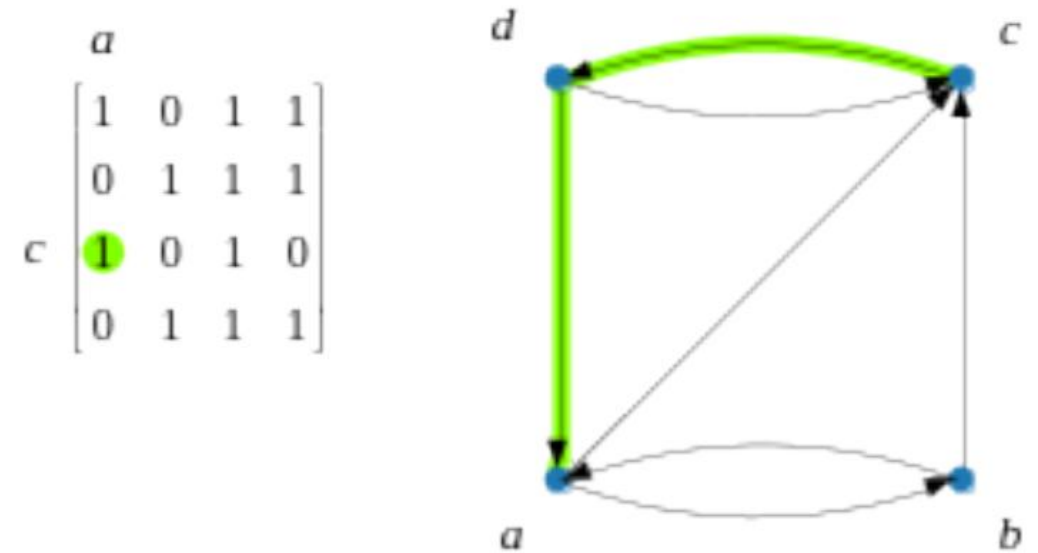


图 6. 节点 c 途径一个节点到达 a 的路径

Directed Graphs Split Degree into Out-Degree and In-Degree

① In-degree: the number of edges pointing into a node; column sums $\mathbf{1}^T \mathbf{A}$ give every node's in-degree

② Out-degree: the number of edges leaving a node; row sums $\mathbf{A} \mathbf{1}$ give every node's out-degree

③ In-degree plus out-degree equals a node's degree; the diagonal of the Gram matrix $\mathbf{A}^T \mathbf{A}$ also gives in-degree, and $\mathbf{A} \mathbf{A}^T$ gives out-degree

④ Note: computing out-degree and in-degree via Gram matrices only works for unweighted directed graphs

Figure 7 · In-Degree of the Four Nodes

- (a)–(d) show the in-degree of nodes a, b, c, d in turn: 2, 1, 3, 1 — matching the diagonal of the Gram matrix $A^T A$

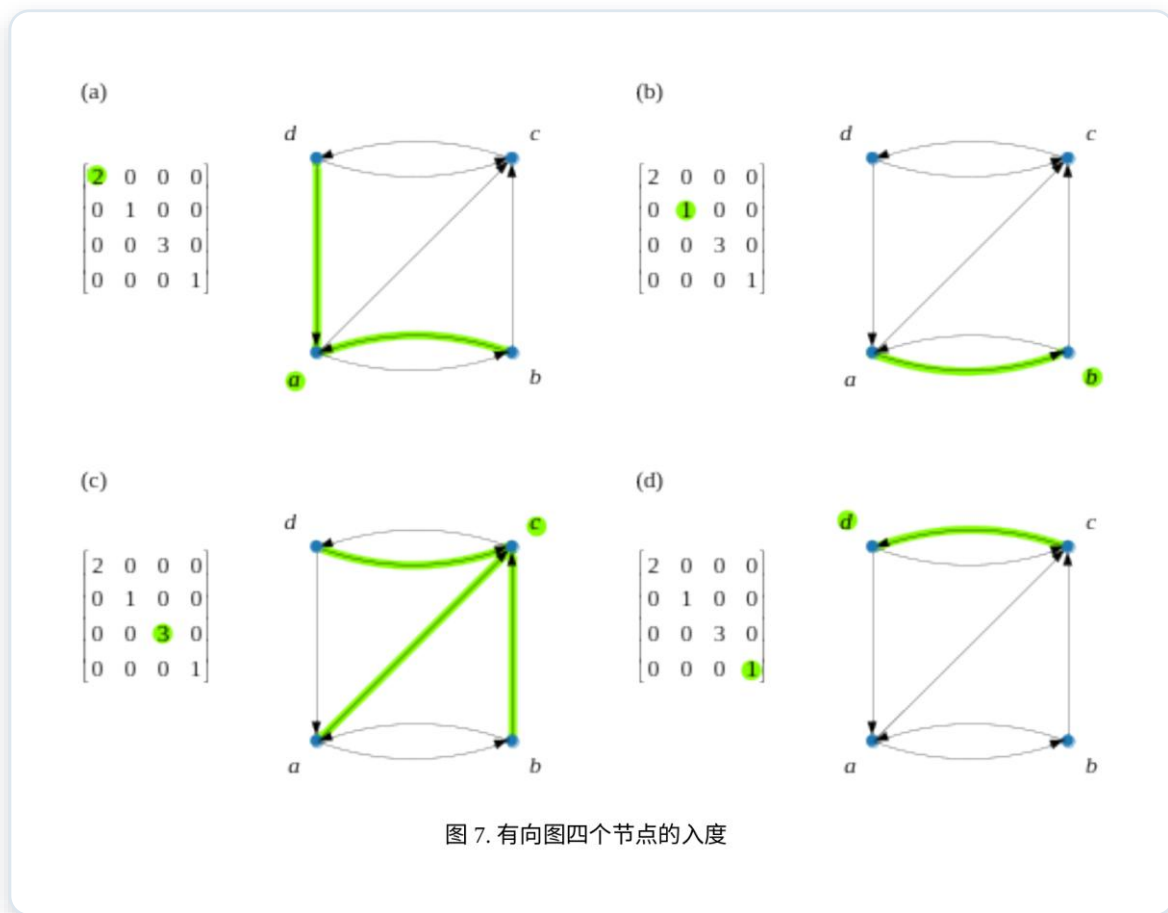


Figure 8 · Out-Degree of the Four Nodes

- (a)–(d) show the out-degree of nodes a, b, c, d in turn: 2, 2, 1, 2 — matching the diagonal of the Gram matrix AA^T

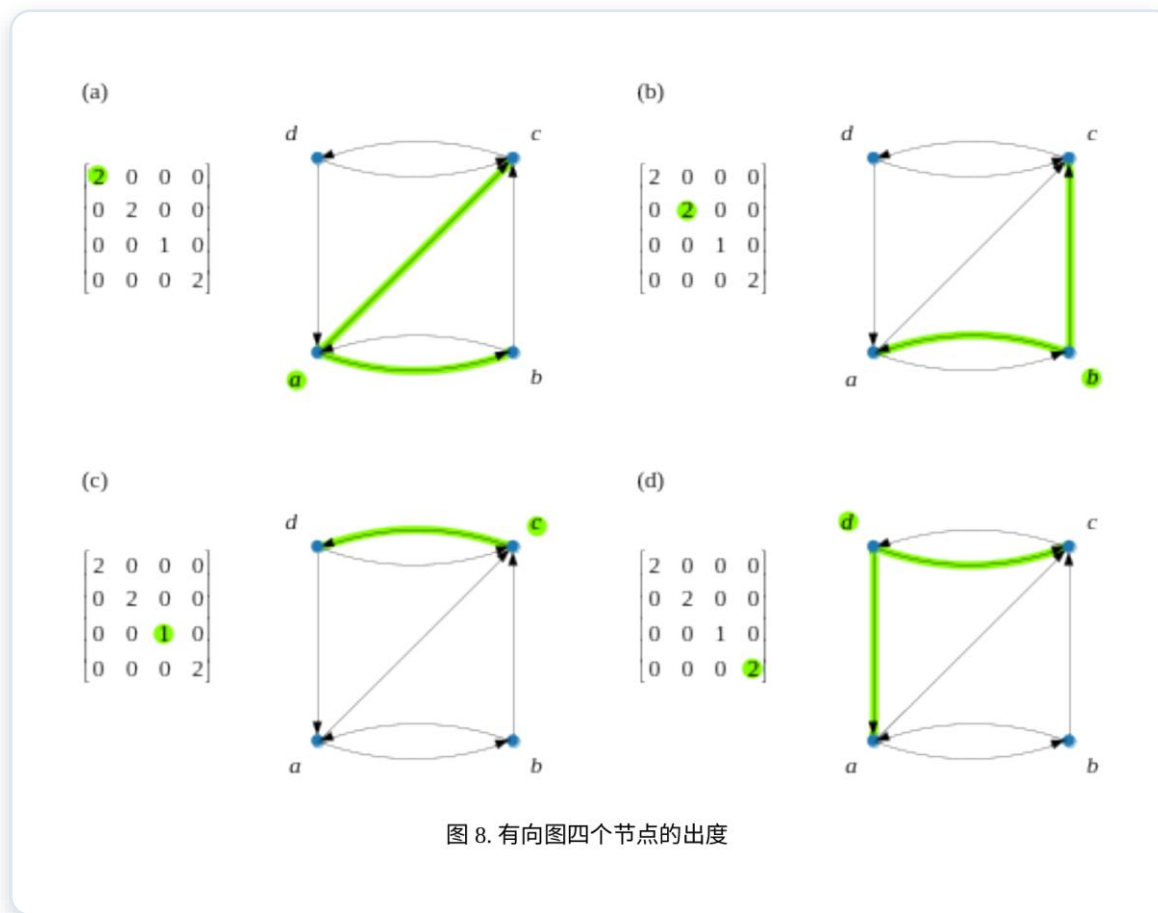


Figure 9 · One Way to Pass the Ball

- Six students a, b, c, d, e, f pass a ball; each pass can go to anyone but themselves. Starting from a, after 4 passes the ball returns to a — how many ways can this happen? The figure shows one such path: $a \rightarrow f \rightarrow e \rightarrow b \rightarrow a$

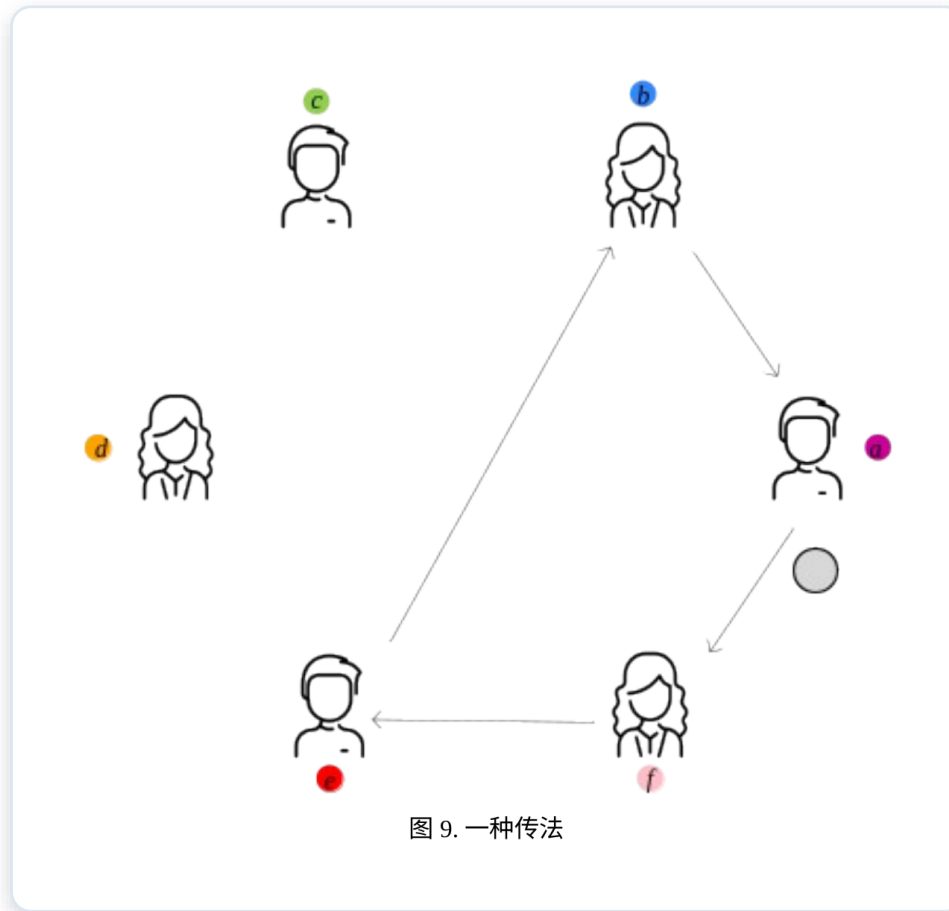


图 9. 一种传法

Figure 10 · A Network of All Possible Paths

- Treat each pass as a jump from one column of nodes to the next; 4 passes give 5 columns of nodes. We need every path in this network that starts at *a* and returns to *a*

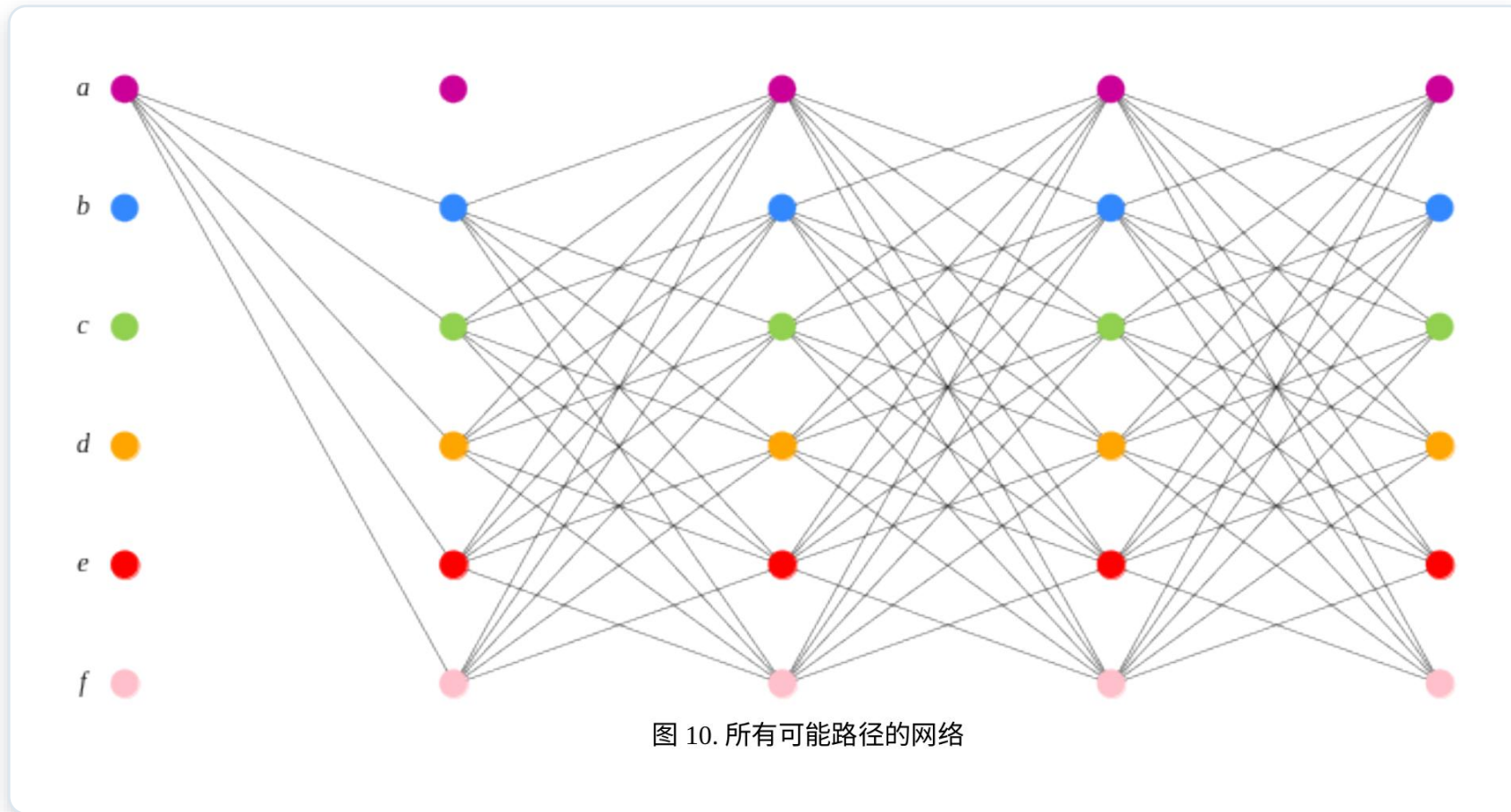


Figure 11 · The Directed Graph Representing the Ball-Passing Problem

- Treat the six students as 6 nodes; the passing rule becomes a directed graph where every node points to the other 5 — the adjacency matrix A is a 6×6 matrix with 0 on the diagonal and 1 everywhere else

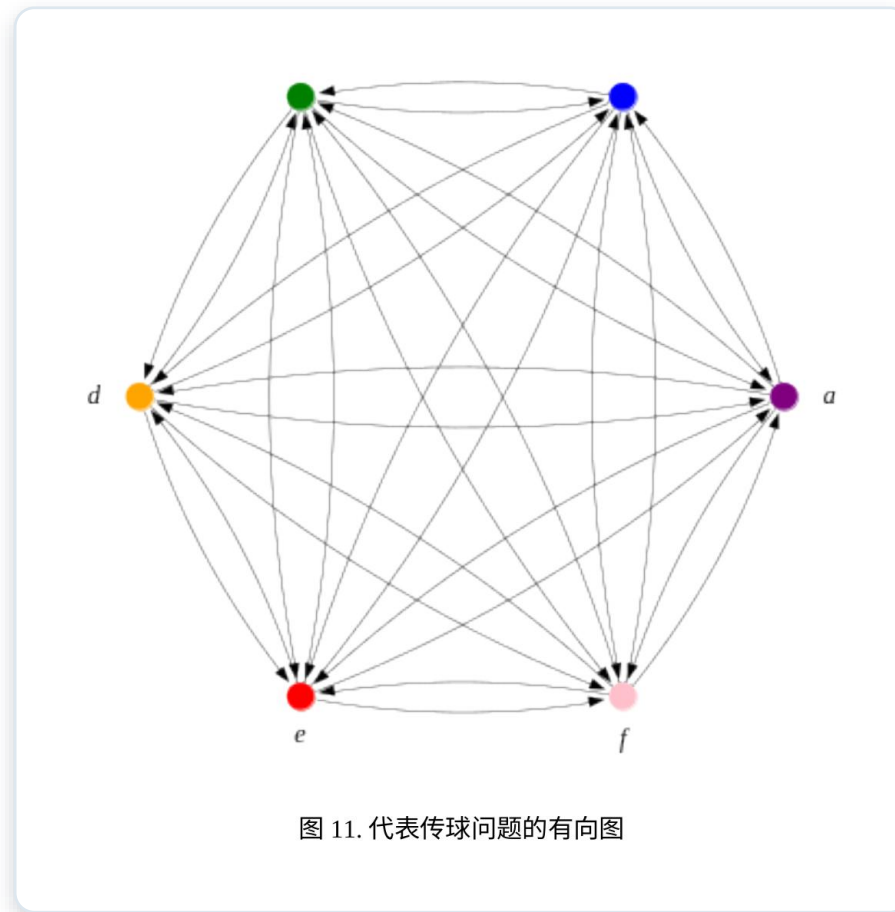


Figure 12 · What the Matrix Multiplication $x_1 = Ax_0$ Means

- The ball starts in a's hands, recorded as state vector x_0 ; $x_1 = Ax_0$ gives the 5 students who might hold the ball after 1 pass — "one becomes five"; x_1 's entries sum to $5 = 5^1$

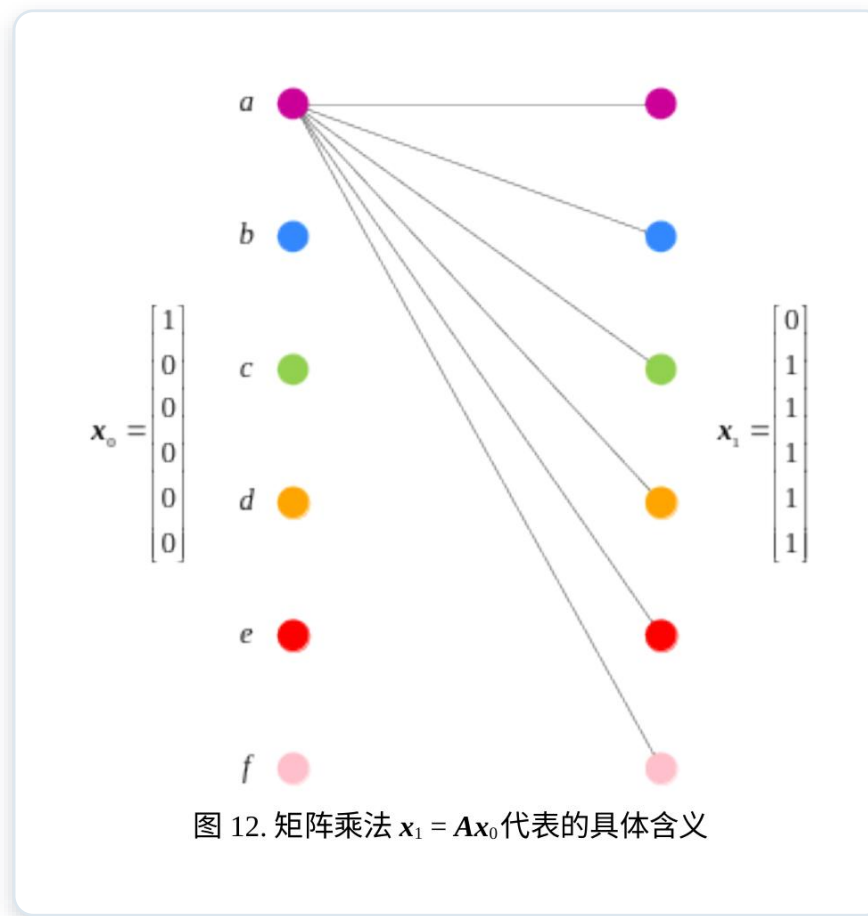


Figure 13 · The Sum of All Entries of Matrix A

- All entries of A sum to 30, i.e., 6×5 : 6 nodes, each with 5 possible passes — also the total number of starting paths for the 2nd pass

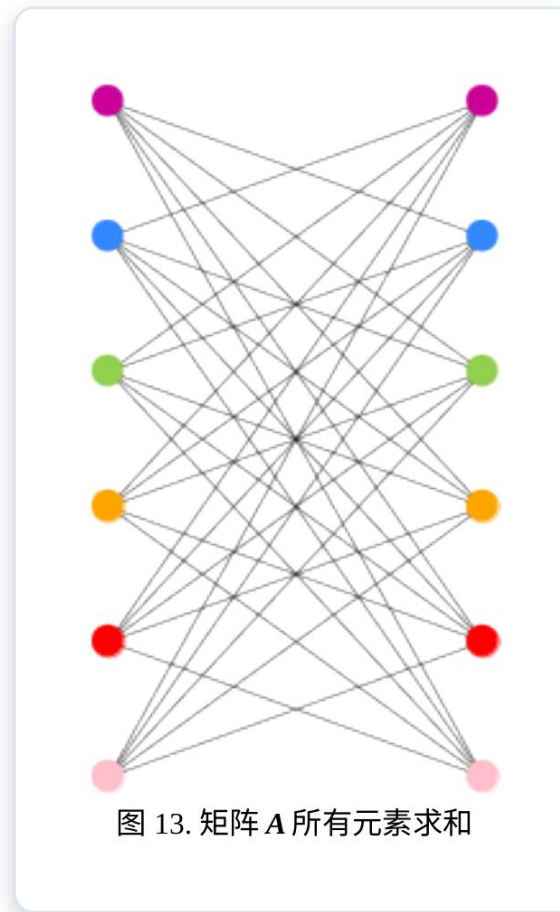


Figure 14 · What the Matrix Multiplication $x_2 = Ax_1$ Means

- $x_2 = Ax_1 = AAx_0$ shows 5 ways the ball returns to a after 2 passes, and 4 ways it lands with b; x_2 's entries sum to $25 = 5^2$, i.e., "one becomes five, five becomes twenty-five"

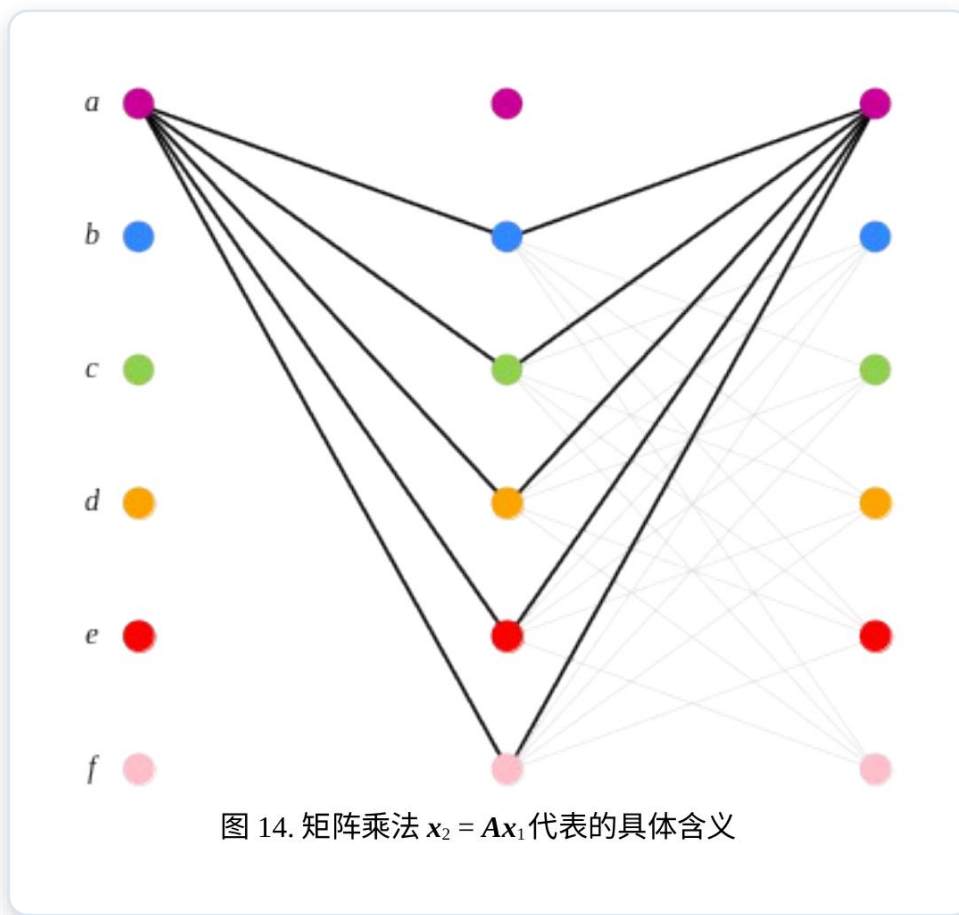
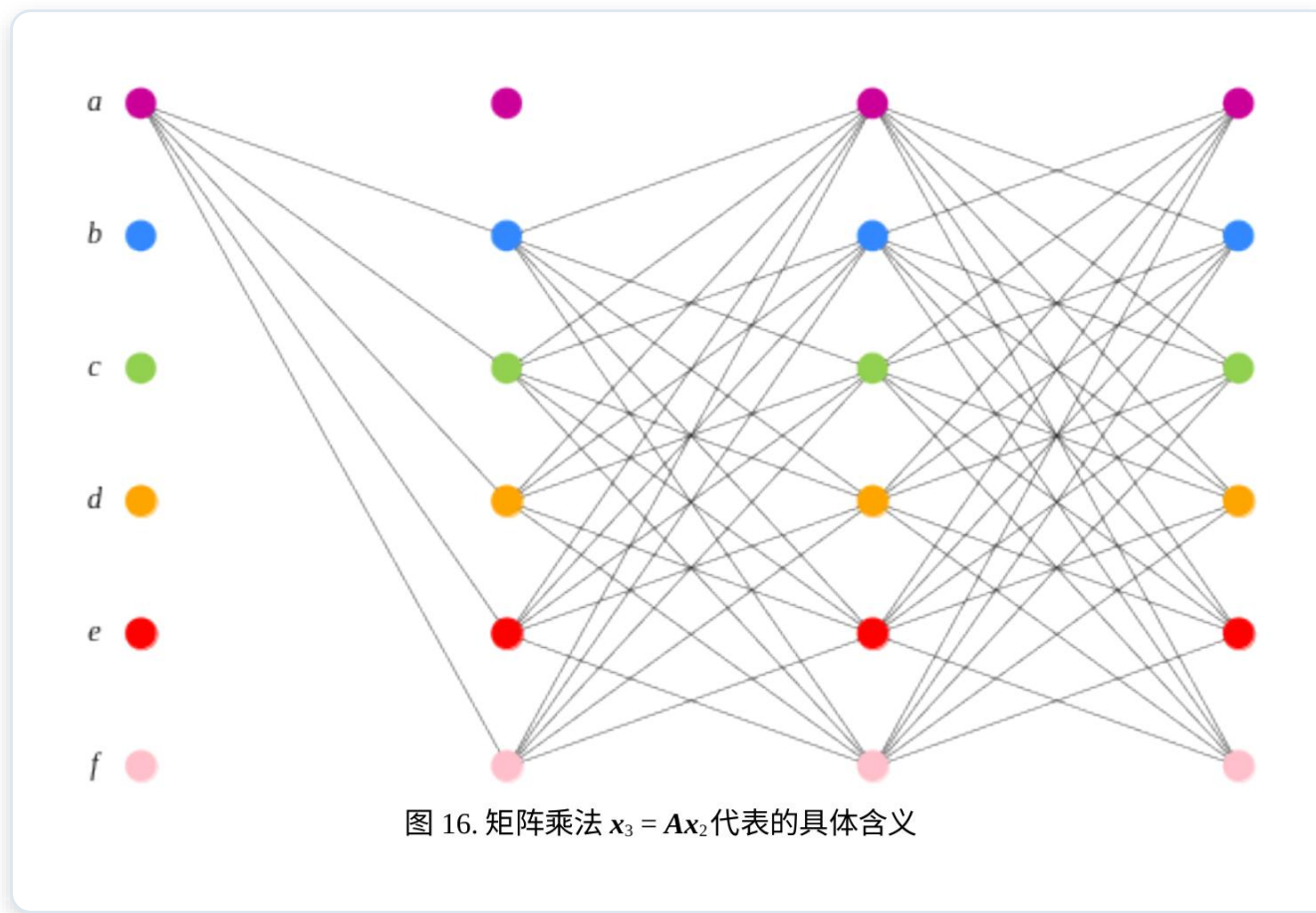


Figure 16 · What the Matrix Multiplication $x_3 = Ax_2$ Means

- $x_3 = Ax_2 = AAx_0$ shows 20 ways the ball returns to a after 3 passes, and 21 ways it lands with each of the other students; A^3 's diagonal is 20, its off-diagonal entries are 21



$\mathbf{x}_4 = \mathbf{A}\mathbf{x}_3 = \mathbf{A}^4\mathbf{x}_0$ Gives the Answer to the Ball-Passing Problem

$$\mathbf{x}_4 = \mathbf{A}\mathbf{x}_3 = \mathbf{A}^4\mathbf{x}_0 = [105 \ 104 \ 104 \ 104 \ 104 \ 104]^T$$

- Starting from a, after 4 passes the ball returns to a in exactly 105 ways — this is the answer to the question posed at the start of the section
- Every other entry of $\mathbf{x}_4$ is 104, the number of ways the ball lands with each other student after 4 passes; $\mathbf{A}^4$'s diagonal is 105, its off-diagonal entries are 104

SUMMARY

Directed Edges Break Symmetry; Out-Degree and In-Degree Split by Row and Column

- ① A directed graph's edges have direction, encoding one-way relationships between nodes
- ② A directed graph's adjacency matrix $\mathbf{A}$ is a square matrix, generally not symmetric; entry (i, j) marks an edge from i to j
- ③ The k -th power of $\mathbf{A}$ counts, entry by entry, the paths from one node to another using exactly k edges
- ④ Out-degree comes from row sums, in-degree from column sums; the two are generally not equal
- ⑤ The diagonals of the Gram matrices $\mathbf{A}^T\mathbf{A}$ and $\mathbf{A}\mathbf{A}^T$ give in-degree and out-degree respectively, but only for unweighted directed graphs
- ⑥ Ball-passing problem: state vectors $\mathbf{x}$ and powers of $\mathbf{A}$ turn combinatorial counting into matrix arithmetic

Key Terms in This Section

Directed Graph *A graph whose edges have direction; connections are generally not symmetric*

Out-Degree *The number of edges leaving a node; the row sum of the adjacency matrix*

State Vector *A vector recording where the ball is (or its distribution) in the ball-passing problem*

Directed Adjacency Matrix *An $n \times n$ square matrix; entry (i, j) marks an edge from i to j*

In-Degree *The number of edges pointing into a node; the column sum of the adjacency matrix*

Ball-Passing Problem *Counting paths that leave a node and return to it after several steps, via matrix powers*

FURTHER READING

Further Reading

Review · 15.1 Undirected Graphs

Compare the symmetric adjacency matrix of undirected graphs with the asymmetric one here

Review · Chapter 3, Block Matrix Multiplication

Understand the computation behind powers A^k , $A^2 A$ of the adjacency matrix

Preview · 15.3 Transition Matrices

Normalize the adjacency matrix by row (or column) to get a matrix describing random walks

Open-Source Code Repository

Code and figures for the whole book: <https://github.com/Visualize-ML>

Exercises Q1 – Q9

Q1 Use NetworkX to build and draw every directed graph in Table 1.

Q3 Compute the out-degree and in-degree of every node in Table 1's directed graphs.

Q5 Use NetworkX to draw the directed graph in Figure 11, and compute its adjacency matrix.

Q7 In Python, verify that in Figure 1 the sum of every node's in-degree equals the sum of every node's out-degree, and both equal the total number of edges.

Q9 Use `numpy.trace()` to compute the trace of $A @ A$ for Figure 1's adjacency matrix A , and explain what this number means in graph terms.

Q2 Compute the adjacency matrix of every directed graph in Table 1.

Q4 4 students pass a ball, again never to themselves. Starting from a , with at most 3 passes, the ball returns to a — compute how many ways this can happen.

Q6 In Python, compute $A @ A @ A$ for Figure 1's adjacency matrix A , and explain what each entry's path count means.

Q8 Design your own directed graph on 5 nodes, draw it with NetworkX, and compute its adjacency matrix, out-degree, and in-degree.

Exercises Q10 – Q17

Q10 In Python, verify that a directed graph's adjacency matrix is symmetric if and only if every edge's reverse edge also exists.

Q12 In Python, compute the Gram matrices $A^T A$ and $A A^T$ for the graph you designed in Q8, and verify their diagonals give in-degree and out-degree.

Q14 Use NetworkX's `in_degree()` and `out_degree()` functions to compute each node's in-degree and out-degree for Figure 1, and compare with your hand calculation.

Q16 Explain why computing out-degree and in-degree via Gram matrices only works for unweighted directed graphs, not weighted ones.

Q11 Explain why a directed graph's adjacency matrix generally does not satisfy A transposed equals A .

Q13 Give real-world examples that can be modeled as directed graphs (e.g. webpage links, social-media follows), and draw the corresponding directed graph.

Q15 Build a weighted version of Figure 1 in Python where edge weights are flight ticket prices, and label the weights with `nx.draw_networkx_edge_labels()`.

Q17 Write a Python function that checks whether a given square matrix is a valid adjacency matrix for a directed graph.

Exercises Q18 – Q25

Q18 For the ball-passing problem of Figure 11, if the rule changes to 5 passes starting and ending at *a*, use Python to compute the number of ways.

Q20 In Python, compare the adjacency matrix of the directed graph you designed in Q8 with the adjacency matrix of the undirected graph obtained by making every edge two-way.

Q22 Explain the practical meaning of out-degree and in-degree in social network analysis — which one corresponds to follower count, and which to following count.

Q24 Based on this section, explain why matrix powers are an important tool for analyzing multi-step paths and multi-step propagation on a graph.

Q19 For the ball-passing problem of Figure 11, use Python to compute how many ways there are for the ball to land with *f* after exactly 4 passes starting from *a*.

Q21 Use Matplotlib to draw a heatmap of the adjacency matrix from Q8, and compare it against the graph's connection structure.

Q23 In Python, verify that for a complete directed graph on *n* nodes where every pair has edges both ways, every node's out-degree and in-degree equal *n*-1.

Q25 Preview Section 15.3, and guess what property the matrix would have if the adjacency matrix were normalized by row; explain your reasoning.

10 Prompts to Explore with AI

1

Explain the essential difference between directed and undirected adjacency matrices, and why a directed graph's adjacency matrix is generally not symmetric.

2

Explain what the powers A^2 and A^3 of the adjacency matrix mean in graph terms.

3

Explain the definitions of out-degree and in-degree, and how they correspond to row sums and column sums of the adjacency matrix.

4

Explain why the diagonals of the Gram matrices $A^T A$ and $A A^T$ give in-degree and out-degree, and why this only works for unweighted directed graphs.

5

Using the ball-passing problem as an example, explain how a state vector and matrix powers turn a combinatorial counting problem into linear algebra.

10 Prompts to Explore with AI

6 Give examples of real networks (webpage links, flight routes, social-media follows) that are naturally modeled as directed graphs.

7 Explain the difference between weighted and unweighted directed graphs, and what kinds of features edge weights can represent.

8 Explain the graph-theoretic meaning of summing all entries of matrix A in the ball-passing problem, and why it equals the number of nodes times the out-degree.

9 Explain why a directed graph's adjacency matrix becomes symmetric — structurally equivalent to an undirected graph — once every edge has its reverse edge present too.

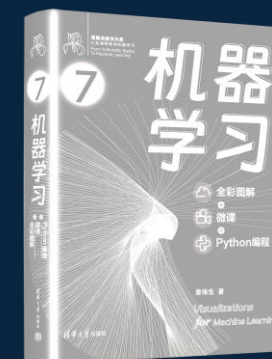
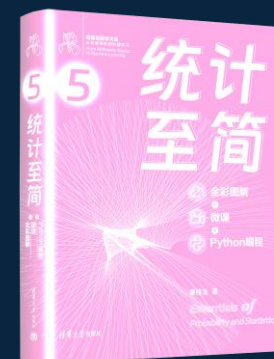
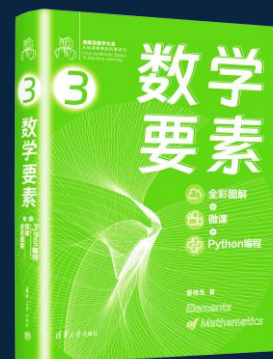
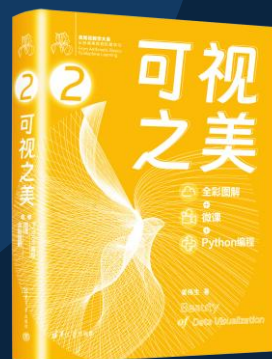
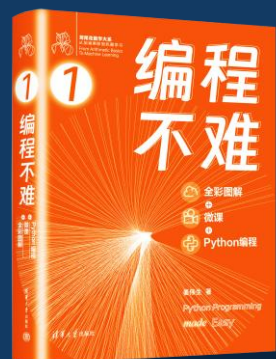
10 Explain the connection between a directed graph's adjacency matrix and the transition matrix covered in Section 15.3, and how the transition matrix is derived from it.

数学

数学基础

线性代数

概率统计



Python编程

数据可视化

工具

数据分析
图和网络

回归、降维
分类、聚类

实践

